Python for Machine Learning

A Complete Learning Journey

Mayur Patil

August 23, 2026

Contents

Part I

Python Fundamentals

Chapter 1

Python Data Types

1.1 Why This Matters

In Machine Learning, your data will take many forms: numerical arrays, text labels, boolean masks, and configurations. Understanding the core Python data types is essential because applying the wrong operation to the wrong type (e.g., trying to add a string to an integer) will crash your pipeline. Knowing how to check types and understand "truthiness" will help you debug faster.

1.2 Explanation

1.2.1 1. Numeric Types

- `int`: Whole numbers. Python handles arbitrarily large integers automatically.
- `float`: Decimal numbers. Standard for ML model weights and probabilities.
- `complex`: Complex numbers (e.g., `3 + 4j`). Rarely used in standard ML, but important for signal processing.

1.2.2 2. Text and Binary Types

- `str`: Text data, enclosed in quotes. Immutable.
- `bytes / bytearray`: Raw byte data. Useful for reading images, audio, or serialized ML models.

1.2.3 3. Boolean and None

- `bool`: `True` or `False`. Used for logic and masking.
- `None`: A special constant representing the absence of a value (similar to `null` in other languages).

1.2.4 4. Collection Types (Brief intro)

- `list`: Ordered, mutable sequence `[1, 2, 3]`.
- `tuple`: Ordered, immutable sequence `(1, 2, 3)`.
- `set`: Unordered collection of unique items {`1, 2, 3`}.
- `dict`: Key-value pairs {`"a": 1, "b": 2`}.

1.2.5 5. `type()` and `isinstance()`

- `type(x)` returns the exact type of `x`.
- `isinstance(x, type)` checks if `x` is an instance of a specific type (or a tuple of types). **This is the preferred, professional way to check types.**

1.2.6 6. Truthiness

In Python, every object can be evaluated as a boolean.

- **Falsy**: `0`, `0.0`, `""` (empty string), `None`, `[]`, {}, `()`, `set()`.
- **Truthy**: Almost everything else!

1.2.7 7. Equality (==) vs Identity (is)

- `==` checks if the **values** are equal.
- `is` checks if they are the **exact same object in memory**.

1.3 Examples

1.3.1 Level 1 — Basic (Type Checking)

```
x = 3.14
print(type(x))                    # Output: <class 'float'>
print(isinstance(x, float))       # Output: True
print(isinstance(x, (int, float))) # Output: True (checks if int OR
    float)
```

1.3.2 Level 2 — Intermediate (Truthiness)

```
empty_list = []
if empty_list:
    print("This won't print because empty lists are falsy.")

if not empty_list:
    print("This WILL print!")  # Preferred way to check if a list is
        empty
```

1.3.3 Level 3 — Hard (Equality vs Identity with None)

```
a = None
b = None

# Because None is a singleton (only one exists in memory), BOTH are
    True.
print(a == b)  # True
print(a is b)  # True

list_1 = [1, 2]
list_2 = [1, 2]
print(list_1 == list_2) # True, they have the same values
print(list_1 is list_2) # False, they are different objects in memory!
```

1.4 Common Mistakes

WARNING:
1. Using `type(x) == list` instead of `isinstance(x, list)`. `isinstance` gracefully handles inheritance (subclasses), which is crucial when working with ML libraries that create custom object types.
2. Checking if a list is empty using `if len(my_list) == 0:`. The Pythonic way is simply `if not my_list:`.

ML CONNECTION:
When reading a CSV dataset using Pandas, missing values often start as `None` or `NaN` (Not a Number, which is a float). Knowing how to detect and handle these null types, and understanding how booleans act as masks (e.g., selecting all rows where age ¿ 30), is the absolute foundation of data preprocessing.

1.5 Exercises

Exercise 1: Truthiness and Types Predict the output of the following code snippet:

```
data = ""
result = 100

if data:
    result = result + 50
else:
    result = result - 20

print(result)
```

Solution: The output is 80. An empty string evaluates to False in Python, triggering the else block.

Exercise 2: Debugging and Best Practices The following function tries to check if the input is a string and if it is not empty. However, it uses poor Python practices. Rewrite it using `isinstance()` and Pythonic truthiness.

```
def process_text(text):
    if type(text) == str:
        if len(text) > 0:
            print("Processing:", text)
        else:
            print("Empty string provided.")
    else:
        print("Error: Input is not a string.")
```

Refactored Solution:

```
def process_text(text):
    if isinstance(text, str):
        if text:
            print("Processing:", text)
        else:
            print("Empty string provided.")
    else:
        print("Error: Input is not a string.")
```

Chapter 2

Strings and Manipulation

2.1 Why This Matters

In Machine Learning and Data Science, a massive portion of data is unstructured text (e.g., medical records, tweets, reviews, log files). To perform Natural Language Processing (NLP) or basic data cleaning, you must know how to manipulate strings efficiently.

2.2 Explanation

A string (`str`) in Python is a sequence of characters used to represent text.

IMPORTANT:
Immutability: Strings are immutable, meaning once created, they cannot be changed in place. Any operation that modifies a string actually creates a brand new string.

2.2.1 1. Indexing and Slicing

Because strings are sequences, you can access individual characters using an index (starting at 0) and slice them to extract substrings using `[start:stop:step]`.

```
text = "Machine Learning"

print(text[0])    # 'M'
print(text[-1])   # 'g' (negative indices count from the end)

# Slicing [start:stop] (Stop is exclusive)
print(text[0:7]) # 'Machine'
print(text[:7])   # 'Machine'
print(text[8:])   # 'Learning'
```

2.2.2 2. Essential String Methods

- `.lower()` / `.upper()`: Changes case.
- `.strip()`: Removes leading and trailing whitespace.
- `.split(delimiter)`: Splits the string into a list.
- `.replace(old, new)`: Replaces substrings.

```
# Cleaning dirty data by chaining methods
raw_data = "   user@email.com   \n"
clean_data = raw_data.strip().lower()
print(clean_data)  # 'user@email.com'
```

2.2.3 3. F-Strings and The Format Mini-Language

F-strings (formatted string literals) are the modern way to inject variables into strings. When inside an f-string's curly braces {}, anything to the **right** of the colon : is the formatting rule.

- {`epoch:02d`} (Integer Padding): Pads the integer with zeros until it is at least 2 characters wide.
- {`loss:.3f`} (Float Rounding): Rounds a floating point number to exactly 3 decimal places.
- {`accuracy:.1%`} (Percentage Conversion): Multiplies the float by 100, rounds to 1 decimal place, and appends a `%` sign.

```
epoch = 5
loss = 0.03456
acc = 0.892

print(f"[Epoch {epoch:02d}] Loss: {loss:.3f} | Acc: {acc:.1%}")
# Output: [Epoch 05] Loss: 0.035 | Acc: 89.2%
```

2.3 Common Mistakes

WARNING:
Trying to change a string character directly like `text[0] = 'X'` will throw a `TypeError`. Also, forgetting that `.strip()` does not remove spaces *inside* the string, only at the edges.

ML CONNECTION:
When preparing text data for an ML model (like a text classifier or an LLM), you usually build a **preprocessing pipeline**. This involves converting all text to lowercase, removing punctuation via `.replace()`, and splitting sentences into individual words via `.split()`.

2.4 Solutions to Exercises

Exercise 1: String Slicing

```
file_path = "/data/images/dataset_2024.csv"
print(file_path[-16:])
```

Solution: Negative indexing ensures we get exactly the last 16 characters regardless of what the parent directories are named.

Exercise 2: Data Cleaning

```
raw_tweet = "   Python is AWESOME for Machine Learning!!!   \n"
clean_tweet = raw_tweet.strip().lower().replace("!!!", "")
print(clean_tweet)
```

Solution: Chaining methods works perfectly here. `python is awesome for machine learning`

Exercise 3: F-Strings and Formatting

```
epoch = 5
loss = 0.0345678
val_accuracy = 0.892
print(f"[Epoch {epoch:02d}] Training Loss: {loss:.3f} | Validation Acc:
    {val_accuracy:.1%}")
```

Solution: `[Epoch 05] Training Loss: 0.035 | Validation Acc: 89.2%`

Chapter 3

Lists

3.1 Why This Matters

In Machine Learning, lists are the absolute foundation for storing sequences of data. Before you learn advanced numerical arrays (like NumPy), you will use standard Python lists to store model predictions, sequences of tokens in NLP, or rows of data from a CSV file.

3.2 Explanation

A list in Python is an ordered, **mutable** (changeable) collection of items. Lists can contain items of different data types, including other lists. They are created using square brackets `[]`.

Because they are ordered sequences, you can index and slice them exactly like Strings. But unlike Strings (which are immutable), you can modify lists *in place*.

3.2.1 1. Indexing, Slicing, and Mutating

```
fruits = ["apple", "banana", "cherry"]

# Indexing and Slicing (Same as strings!)
print(fruits[0])     # 'apple'
print(fruits[-1])    # 'cherry'
print(fruits[:2])    # ['apple', 'banana']

# Mutating (Changing items in place)
fruits[1] = "blueberry"
print(fruits)        # ['apple', 'blueberry', 'cherry']
```

3.2.2 2. Essential List Methods

- `.append(item)`: Adds an item to the **end** of the list.
- `.insert(index, item)`: Inserts an item at a specific position.
- `.pop(index)`: Removes and returns the item at the index (defaults to the last item if no index is given).
- `.remove(value)`: Removes the *first* occurrence of a specific value.
- `.extend(list2)`: Adds all elements of a second list to the end of the first.
- `.sort()`: Sorts the list in place (modifies the original list).

3.2.3 3. Nested Lists (2D Data)

Lists can contain other lists. This is how we represent 2D data (like a spreadsheet or an image) in pure Python.

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

# To get the number 6:
# First get the middle row (index 1), then the last item (index -1)
print(matrix[1][-1]) # 6
```

3.3 Common Mistakes: The Aliasing Trap

WARNING:
This is the most common mistake made by Python beginners. Because lists are mutable, variables simply **point** to the list in memory.
If you write `b = a`, you did not copy the list. You created a second variable pointing to the same list. Modifying `b` will modify `a`.

To make an actual independent copy, use the `.copy()` method or slice it `[:]`:

```
a = [1, 2, 3]
c = a.copy()
c.append(5)
print(a)        # Still [1, 2, 3], 'c' is independent.
```

ML CONNECTION:
When building neural networks from scratch, the architecture is often defined as a list of layers. Furthermore, when reading raw data before passing it to a library like Pandas, it usually starts as a "list of lists" where each inner list represents a single row in a dataset.

3.4 Solutions to Exercises

Exercise 1: The Queue

```
queue = ["img1.png", "img2.png"]
queue.append("img3.png")
print(queue.pop(0))
```

Solution: `.append("img3.png")` *safely adds the new image to the very end of the list.* `.pop(0)` *removes the element at index 0 (the first element) and returns it.*

Exercise 2: Nested Data Extraction

```
batch = [
    [ [0, 0], [0, 0] ],
    [ [1, 1], [1, 1] ],
    [ [2, 2], [2, 9] ]
]
print(batch[2][1][1])
```

Solution: `batch[2]` *selects the 3rd image.* `[1]` *selects the 2nd row* `[2, 9]`*.* `[1]` *selects the 2nd element* `9`*.*

Exercise 3: The Copy Trap

```
original_data = [100, 200, 300]
normalized = original_data.copy()
normalized.remove(300)

print("Original:", original_data)
print("Normalized:", normalized)
```

Solution: By using `.copy()`*, Python creates a brand new list in memory. Now* `normalized` *points to this new list, while* `original_data` *points to the old one.*